PROJECT REPORT

Automated Web Scraping & CRM Data Synchronization

Submitted by: **Aditya Mishra**

BCA Student, Lovely Professional University (Expected 2027)

Date: 11 November 2026

GitHub Repository: https://github.com/Adi-dot0/automated-web-scraping-crm-sync

1. Abstract

This project automates the collection of product data from a website and its synchronization with a CRM system. A Python scraper collects records page by page from books.toscrape.com, a public website built specifically for scraping practice. The data is validated and formatted (price parsing, rating conversion, duplicate removal), stored in a database, and prepared for synchronization to HubSpot CRM as products, with a duplicate check before creating any record. A Streamlit dashboard monitors scraping and sync status. In the verified live run, the system scraped 60 real records, stored all 60, and on a second run created 0 duplicates - all 60 records were correctly recognized as updates.

2. Business Problem

Businesses collect product information, pricing updates and competitor data from websites manually. Manual collection wastes time and produces outdated, inconsistent data. Automating collection, validation and CRM synchronization keeps business data updated without repeated manual work.

3. Objectives

- Collect structured data from website pages automatically, including pagination
- Validate and format scraped data and remove duplicate records
- Store scraped data in a structured database with sync history
- Synchronize validated records with HubSpot CRM without creating duplicates
- Monitor scraping status, CRM sync status and data quality in a dashboard
- Follow responsible scraping practice: permitted target site, polite request rate, self-identifying user agent

4. Technology Stack

Tool	Use in this project
Python	Automation and pipeline logic
Requests + BeautifulSoup	Web scraping of HTML pages
Selenium	Ready module for JavaScript-heavy websites (not needed for the demo site)
Pandas	Data handling and CSV exports
SQLite	Storage of scraped records, sync history and duplicate log
HubSpot API	CRM synchronization (products), live mode with a private-app token
Streamlit	Monitoring dashboard
GitHub	Version control and project hosting

5. System Architecture

Website Pages → Scraper (Requests + BeautifulSoup) → Validation and Formatting → Deduplication → Database Storage → CRM Synchronization (HubSpot) → Monitoring Dashboard (Streamlit)

6. Modules of the System

Module 1 - Website Scraping (scrapers/)

website_scraper.py sends requests with a self-identifying user agent, follows the catalogue pagination (page-1, page-2, ...), waits one second between pages, and parses each product card into a raw record: title, price text, rating, availability and product URL. selenium_handler.py provides the browser-automation pattern for websites that load data with JavaScript.

Module 2 - Data Validation and Formatting (processing/data_validation.py)

Prices are parsed from text such as a pound amount into numbers, star ratings given as words (One to Five) become numbers 1 to 5, availability is standardized to In stock or Out of stock, and records missing a title, price or URL are rejected. Duplicates are detected by product URL, which is the unique key of a record.

Module 3 - Database Storage (database/)

Three tables: scraped_records (one row per product, product_url unique), sync_history (every sync run with counts and status) and duplicate_log. Storing uses the unique key: a new URL is inserted, an existing URL is updated with the latest price and rating, so re-scraping never creates duplicates.

Module 4 - CRM Synchronization (crm/)

hubspot_api.py maps each record to a HubSpot Product (name, price, and a description containing rating, availability and the source URL) and searches HubSpot by source URL before creating, so records are updated rather than duplicated. crm_sync.py runs in two modes: live mode when a HubSpot private-app token is provided through an environment variable, and dry-run mode by default, in which no data leaves the machine and a hubspot_import.csv file is produced for manual import. Both modes are recorded in sync_history.

Module 5 - Monitoring Dashboard (dashboard/app.py)

A Streamlit dashboard reads the database and shows total records, sync runs and last sync status, duplicates logged, the rating distribution, a price summary, and the full record and sync-history tables.

7. How to Run the Project

```
pip install -r requirements.txt

python main.py

streamlit run dashboard/app.py (monitoring dashboard)
```

For live HubSpot sync, set the environment variable HUBSPOT_ACCESS_TOKEN to a private-app token from your own free HubSpot account, then run main.py again. The token is

never stored in the code.

8. Testing and Results

Unit tests

Six unit tests cover: parsing a saved HTML page fixture (3 products, correct fields), price cleaning, validation with planted duplicate and invalid rows, the HubSpot payload mapping, and database insert-then-update behaviour. Tests run offline. All 6 tests pass.

Live run results (11 November 2026)

The pipeline was run live against books.toscrape.com, a website created for scraping practice, collecting pages 1 to 3:

Measure	Result
Records scraped (3 pages)	60
Valid after validation	60 (0 invalid, 0 duplicates - site data is clean)
Stored in database (run 1)	60 new
Second run	0 new, 60 updated - no duplicates created
CRM sync	DryRun mode: 60 records prepared, hubspot_import.csv written
Ratings distribution	1 star: 15, 2 star: 8, 3 star: 13, 4 star: 10, 5 star: 14
Price range (GBP)	12.84 to 57.31, average 35.00
Example record	A Light in the Attic - GBP 51.77 - rating 3

The second run is the important verification: a synchronization system that cannot recognize existing records would double the database and the CRM. Here the unique-key check worked live - 60 updates, 0 duplicates, and both runs appear in sync_history.

9. Responsible Scraping

The target website is a sandbox created for scraping practice. The scraper identifies itself in its user agent, waits between page requests, and collects only public product listing data, not personal data. For any real website, the same checks apply first: read the site's terms and robots rules and scrape only what is permitted.

10. Limitations

- Live HubSpot synchronization requires the user's own free HubSpot account and private-app token; the verified runs used dry-run mode, in which nothing is sent to HubSpot

- The Selenium module is provided for JavaScript-heavy sites but is not exercised by the demo site, which is plain HTML

- The parser is written for the demo site's page structure; another website needs its selectors updated

- Prices are in GBP because the demo site prices in pounds

11. Future Scope

Connect a live HubSpot account and verify create/update against the real API; add scheduling so scraping runs daily; extend the scraper to a second permitted source; and add the dashboard to the deployed portfolio alongside Projects 1 and 2.

12. Conclusion

The system completes the full path from public web pages to CRM-ready data: automated collection with pagination, validation and formatting, duplicate-safe database storage, HubSpot synchronization with duplicate checking in both dry-run and live modes, and a monitoring dashboard. Live verification on a real (practice) website showed 60 records collected and stored, and a second run that updated all 60 without creating a single duplicate. The complete project is available on GitHub at https://github.com/Adi-dot0/automated-web-scraping-crm-sync

Appendix - Project Files

main.py; scrapers/website_scraper.py and selenium_handler.py; processing/data_validation.py; database/schema.sql and db.py; crm/hubspot_api.py and crm_sync.py; dashboard/app.py; config/config.json; reports/scraped_products.csv and hubspot_import.csv; tests (3 test files + HTML fixture); requirements.txt; README.md.